Innovation Centre for Education

NCS
NihonCommunicationSolutions

YENEPOYA
(Deemed to be University)

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

A Constituent Unit of Yenepoya (Deemed to be University)

Balmatta, Mangalore - 575002

A Low-Level Design
On
HARDENED CLOUD INFRASTRUCTURE FOR RETAIL DATA

Student Name: Bhavana Pradeep

Industry Mentor: Mr.Shashank

Guided By: Ms.Munisah

TABLE OF CONTENTS

1. INTRODUCTION 1

1.1 Purpose of the Document 1

1.2 Scope 1

2. DETAILED COMPONENT/MODULE DESIGN 2

2.1 Module Name: Core Cryptographic Engine 2

2.1.1 Overview and Responsibilities 2

2.1.2 Class Diagram / Internal Structure 2

2.1.3 Methods and Functions 3

2.2 Module Name: Authentication Module 4

2.2.1 Overview and Responsibilities 4

2.3 Module Name: File Management Module 6

2.3.1 Overview and Responsibilities 6

3. DATABASE AND STORAGE DESIGN 8

3.1 Entity-Relationship (ER) Diagram 8

4. INTERFACE AND API SPECIFICATIONS 10

4.1 Internal APIs (Inter-module Communication) 10

4.2 External APIs (REST) 10

4.2.1 Endpoint URI and HTTP Methods 10

4.2.2 Request Headers and Payload Schemas 11

4.2.3 Response Formats and Status Codes 11

4.2.4 Rate Limiting and Timeouts 12

5. STATE, SESSION, AND CACHE IMPLEMENTATION 13

5.1 Session Data Structure 13

5.2 Caching Strategy 13

5.3 State Transition Diagrams 13

6. SECURITY IMPLEMENTATION DETAILS 14

6.1 Authentication Tokens and Validation Logic 14

6.2 Authorization and Role-Based Access Control (RBAC) Matrix 14

6.3 Data Encryption (At Rest and In Transit) 15

6.4 Input Validation and Sanitization Rules 16

7. ERROR HANDLING, LOGGING, AND MONITORING 17

7.1 Standardized Error Message Formats 17

7.2 Application Logging Levels and Structure 17

8. CONFIGURATION AND ENVIRONMENT SETUP 18

8.1 Environment Variables and Configuration Files 18

8.2 Feature Flags 18

1. INTRODUCTION

1.1 Purpose of the Document

The purpose of this Low-Level Design (LLD) document is to provide a comprehensive, granular blueprint for the implementation of the Hardened Cloud Infrastructure for Secure Retail Data Storage system. While the High-Level Design (HLD) outlines the general architecture and data flow, this LLD specifies the internal logic, class structures, component interfaces, and exact data schemas required by the development team. It serves as the primary technical reference for building the Authentication Module, the Core Cryptographic Engine, the File Management Module, the Intrusion Prevention System, the Admin Audit Module, and the Frontend Interface.

1.2 Scope

The scope of this document encompasses the detailed technical design of the following core modules:

- Authentication Module (auth/): Flask route handlers for user registration, login, and logout. Includes Bcrypt password hashing logic and Flask-Login session establishment.
- Core Cryptographic Engine (security/core_crypto.py): Internal class structure, PBKDF2 key derivation, AES-256-GCM encryption and decryption routines, and exception handling.
- File Management Module (files/): Route handlers for upload, download, and delete. Includes UUID assignment, SHA-256 integrity hashing, and cryptographic shredding.
- Intrusion Prevention System (IPS): Login attempt monitoring, failed attempt threshold logic, account lockout mechanism, and lockout recovery.
- Admin Audit Module (admin/): Route handlers for Admin Dashboard and User Management panel.
- Database Layer (models.py): SQLAlchemy ORM model definitions, relationships, and index specifications.
- Frontend Interface (templates/, static/): Jinja2 template structure, JavaScript AJAX patterns, and form validation logic.

2. DETAILED COMPONENT/MODULE DESIGN

This section provides the low-level architecture for all primary modules of the system. Each module subsection follows a consistent structure: Overview and Responsibilities, Class Diagram/Internal Structure, Methods and Functions (Signatures, Inputs, Outputs), Core Algorithms and Business Logic, and Internal Exception and Error Handling.

2.1 Module Name: Core Cryptographic Engine (security/core_crypto.py)

2.1.1 Overview and Responsibilities

The Core Cryptographic Engine is the security nucleus of the entire application. Its sole responsibility is to perform AES-256-GCM encryption and decryption of arbitrary byte streams using a password-derived key. It operates entirely in memory, accepting byte input and returning byte output. It has no database dependencies and no Flask dependencies, ensuring it remains a pure, independently testable cryptographic module.

The engine enforces the following invariants: every encryption operation generates a new cryptographically random salt and nonce; the derived key material is never stored or logged; and decryption failure due to authentication tag mismatch raises an explicit exception rather than returning corrupt data silently.

2.1.2 Class Diagram / Internal Structure

Class: CoreCryptoEngine

Element	Type	Details
ITERATIONS	Class Constant	Integer: 600,000 — PBKDF2 iteration count
KEY_LENGTH	Class Constant	Integer: 32 — AES-256 key length in bytes
SALT_LENGTH	Class Constant	Integer: 32 — Random salt length in bytes
NONCE_LENGTH	Class Constant	Integer: 12 — AES-GCM nonce length in bytes (96-bit)
_derive_key(password, salt)	Private Static Method	Derives a 256-bit AES key from password and salt using PBKDF2HMAC-SHA256
encrypt_data(data, password)	Public Static Method	Encrypts plaintext bytes; returns (ciphertext, salt_hex, nonce_hex)
decrypt_data(ciphertext, password, salt_hex, nonce_hex)	Public Static Method	Decrypts ciphertext bytes; returns plaintext bytes or raises ValueError

2.1.3 Methods and Functions

Method: _derive_key(password: str, salt: bytes) → bytes

Parameter	Type	Description
password	str	The user-supplied encryption or decryption password
salt	bytes	32-byte cryptographically random salt value
Return Value	bytes	32-byte AES-256 symmetric key derived via PBKDF2HMAC

Internal logic: Instantiates PBKDF2HMAC using algorithm=hashes.SHA256(), length=32, salt=salt, iterations=600000. Calls kdf.derive(password.encode('utf-8')) and returns the resulting bytes.

Method: encrypt_data(data: bytes, password: str) → tuple[bytes, str, str]

Parameter	Type	Description
data	bytes	The raw plaintext byte stream to be encrypted
password	str	The user-supplied encryption password
Return: ciphertext	bytes	The AES-GCM encrypted byte stream including the GCM authentication tag
Return: salt_hex	str	64-character hexadecimal string representation of the random salt
Return: nonce_hex	str	24-character hexadecimal string representation of the random nonce

Encryption sequence:

1. Generate salt = os.urandom(32)
2. Generate nonce = os.urandom(12)
3. Call _derive_key(password, salt) to obtain key bytes
4. Instantiate AESGCM(key)
5. Call aesgcm.encrypt(nonce, data, None) to produce ciphertext with appended GCM tag
6. Return (ciphertext, salt.hex(), nonce.hex())

Method: decrypt_data(ciphertext: bytes, password: str, salt_hex: str, nonce_hex: str) → bytes

Decryption sequence:

- Decode: salt = bytes.fromhex(salt_hex)
- Decode: nonce = bytes.fromhex(nonce_hex
- Call _derive_key(password, salt) to re-derive the identical AES key
- Instantiate AESGCM(key)
- Call aesgcm.decrypt(nonce, ciphertext, None)
- If the GCM authentication tag is invalid, cryptography.exceptions.InvalidTag is caught and re-raised as ValueError('Decryption failed: invalid password or file tampered')
- If successful, return the plaintext bytes

2.1.4 Core Algorithms and Business Logic

The PBKDF2 iteration count of 600,000 introduces approximately 0.3 to 0.8 seconds of computation time per key derivation. This bounds the attack rate of offline brute-force attacks to fewer than three attempts per second on modern hardware. The GCM authentication tag (16 bytes appended to the ciphertext) is verified during decryption. Even if an attacker modifies a single bit of the ciphertext on disk, decryption will fail with an InvalidTag exception rather than producing corrupt data.

2.1.5 Internal Exception and Error Handling

- cryptography.exceptions.InvalidTag: Caught internally and re-raised as ValueError with the message 'Decryption failed: invalid password or file tampered.'
- ValueError from bytes.fromhex(): Raised if salt_hex or nonce_hex contains non-hexadecimal characters. Propagated to the calling route handler.
- TypeError: Raised if data or password is None. The route handler validates these inputs before calling the engine.

2.2 Module Name: Authentication Module (auth/)

2.2.1 Overview and Responsibilities

The Authentication Module manages all aspects of user identity within the system. It implements user registration with Bcrypt password hashing, user login with credential verification and IPS enforcement, Flask-Login session establishment and teardown, and the Flask-Login user_loader callback. No plaintext passwords are ever stored or logged.

2.2.2 IPS Core Algorithm

The Intrusion Prevention System is implemented as database-persisted state on the User model. This design ensures that account lockouts survive server restarts, session clears, and IP address changes. The lockout duration is 15 minutes. The threshold is 5 consecutive failed attempts. A successful login always resets the failed attempt counter and clears any existing lockout.

IPS State Machine:

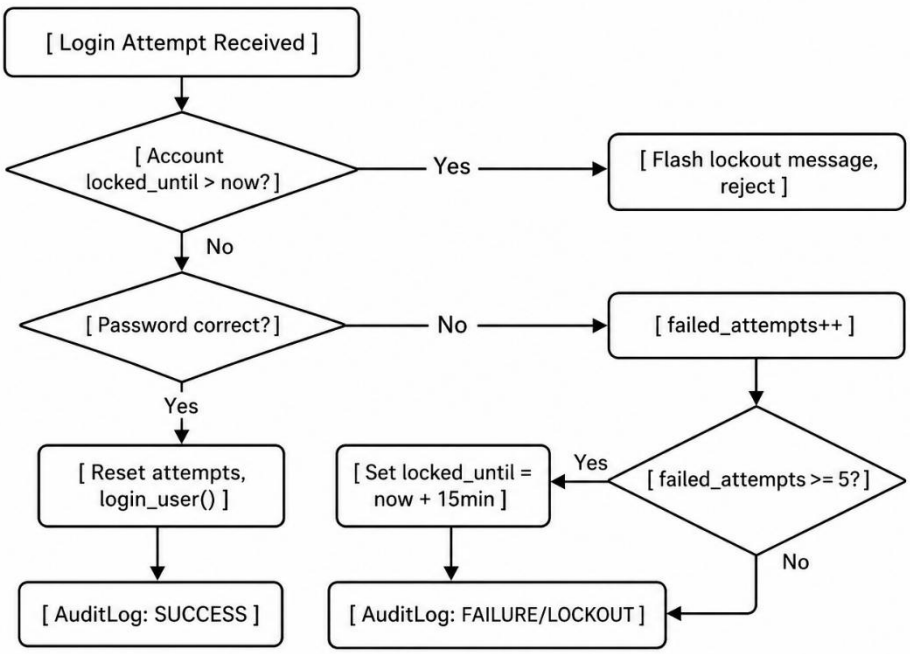


Figure 2.1 — IPS Account Lockout State Machine

2.2.3 Internal Exception and Error Handling

- SQLAlchemy OperationalError: Database write failures during registration or login are caught at the application level and result in a 500 server error response.
- bcrypt.exceptions.CheckError: If the password_hash column contains a corrupted value, CheckError is caught and treated as an authentication failure.

2.3 Module Name: File Management Module

2.3.1 Overview and Responsibilities

The File Management Module handles all file lifecycle operations: upload (encryption), download (decryption), and deletion (cryptographic shredding). It orchestrates calls between the Flask request context, the Core Cryptographic Engine, the file system, and the SQLAlchemy database layer. All routes in this module require an authenticated session.

2.3.2 Methods and Functions — upload()

- Verify current_user.is_authenticated via Flask-Login decorator.
- Extract uploaded file object from request.files['file']. Raise 400 if absent.
- Extract encryption_password from request.form. Raise 400 if absent or empty.
- Read file bytes: file_bytes = file.read().
- Compute integrity hash: file_hash = hashlib.sha256(file_bytes).hexdigest().
- Call CoreCryptoEngine.encrypt_data(file_bytes, encryption_password) to obtain (ciphertext, salt_hex, nonce_hex).

2.3.3 Cryptographic Shredding Algorithm — delete()

The cryptographic shredding algorithm addresses the forensic recovery threat. When a file is conventionally deleted using os.remove(), the OS marks the inode as free but does not zero the underlying disk sectors. Forensic tools can recover this data.

- Query FileRecord by file_id. Verify user_id matches current_user.id. Return 403 if mismatch.
- Construct file path from stored UUID filename.
- Read ciphertext size: file_size = os.path.getsize(path).
- Open file for writing: with open(path, 'wb') as f: f.write(os.urandom(file_size)).
- Call os.remove(path) to remove the now-overwritten file.
- Delete FileRecord from database. Commit.
- Write AuditLog(action='DELETE', status='SUCCESS'). Commit.
- Return JSON {status: 'success'}.

•

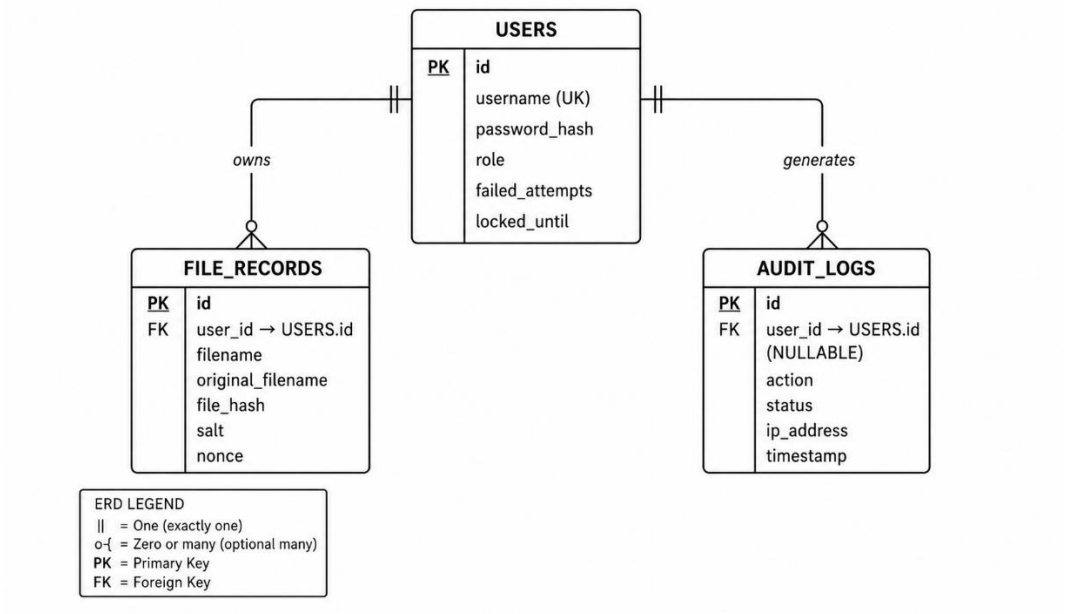
2.3.4 Internal Exception and Error Handling

- FileNotFoundError: Raised if the ciphertext file is missing from the expected path. Caught and returned as 404.
- ValueError from CoreCryptoEngine.decrypt_data(): Raised on wrong password or tampered ciphertext. Caught and returned as 400 with message 'Incorrect decryption password or file integrity failure.'
- PermissionError: Raised if the server process lacks write permissions. Caught and returned as 500.
- IOError/OSError: Caught around all file system operations and returned as 500 with AuditLog entry status='FAILURE'.

3. DATABASE AND STORAGE DESIGN

3.1 Entity-Relationship (ER) Diagram

The system database uses SQLite with three core tables: users, file_records, and audit_logs. Relationships are enforced at both the database level (foreign key constraints) and the application level (ownership filter on every query).



3.2 Indexes and Performance Tuning

SQLAlchemy automatically creates an index on all primary key and unique constraint columns. The following additional indexes are recommended for query performance:

```
CREATE INDEX IF NOT EXISTS idx_file_records_user_id ON file_records(user_id);
CREATE INDEX IF NOT EXISTS idx_audit_logs_user_id ON audit_logs(user_id);
CREATE INDEX IF NOT EXISTS idx_audit_logs_timestamp ON audit_logs(timestamp
DESC);
CREATE INDEX IF NOT EXISTS idx_users_username ON users(username);
```

These indexes support the most frequent query patterns: fetching all FileRecords for a user, fetching AuditLog entries ordered by timestamp, and user lookup by username during login.

3.3 Data Access Objects (DAO) / ORM Mapping

Logical DAO	Responsibilities	Main Tables
UserDAO	Register user, find by username, update failed_attempts, set locked_until, reset lock	users
FileRecordDAO	Create record on upload, query by file_id + user_id, delete on shred	file_records
AuditLogDAO	Append log entry on every user action, query all logs for admin view	audit_logs

3.4 Triggers, Stored Procedures, or Background Jobs

SQLite does not support stored procedures. No database triggers are implemented in the current scope. Account lockout expiry is evaluated at login time by comparing locked_until with the current UTC time — there is no background job to clear expired locks.

4. INTERFACE AND API SPECIFICATIONS

4.1 Internal APIs (Inter-module Communication)

Internal Interface	Caller Module	Callee Module	Communication Pattern
CoreCryptoEngine.encrypt_data()	files/ upload route	security/core_crypto.py	Direct Python method call; in-process
CoreCryptoEngine.decrypt_data()	files/ download route	security/core_crypto.py	Direct Python method call; in-process
db.session.query()	All route handlers	SQLAlchemy ORM / SQLite	ORM method calls within request context
login_user() / logout_user()	auth/ route handlers	Flask-Login	Direct function calls from Flask-Login library
current_user	All protected routes	Flask-Login	Thread-local proxy object populated per request

4.2 External APIs (REST)

4.2.1 Endpoint URI and HTTP Methods

URI	HTTP Method	Auth Required	Role
/auth/register	GET, POST	No	Public
/auth/login	GET, POST	No	Public
/auth/logout	GET	Yes	User, Admin
/dashboard	GET	Yes	User, Admin
/files/upload	POST	Yes	User, Admin
/files/download/<file_id>	POST	Yes	User, Admin
/files/delete/<file_id>	POST	Yes	User, Admin
/admin/dashboard	GET	Yes	Admin only
/admin/users	GET	Yes	Admin only
/admin/users/<user_id>/delete	POST	Yes	Admin only

4.2.2 Request Headers and Payload Schemas

POST /files/upload — Payload Schema:

Field	Type	Required	Description
file	File (multipart)	Yes	The file to be encrypted and stored
encryption_password	String (form field)	Yes	The user-supplied password for key derivation

POST /files/download/<file_id> — Payload Schema:

Field	Type	Required	Description
decryption_password	String (form field)	Yes	The password used when the file was originally encrypted

4.2.3 Response Formats and Status Codes

Scenario	HTTP Status	Response Body
File upload success	200 OK	JSON: {status: 'success', file_id: integer, message: 'File encrypted and stored successfully'}
File download success	200 OK	Binary stream with Content-Disposition: attachment; filename='original_filename'
Wrong decryption password	400 Bad Request	JSON: {status: 'error', message: 'Incorrect decryption password or file integrity failure'}
Unauthorised access attempt	403 Forbidden	Redirect to /dashboard with flash message 'Access denied'
File not found	404 Not Found	JSON: {status: 'error', message: 'File record not found'}
Login success	302 Redirect	Redirect to /dashboard
Login failure	302 Redirect	Redirect to /auth/login with flash message 'Invalid credentials'
Account locked	302 Redirect	Redirect to /auth/login with flash message including remaining lockout duration

4.2.4 Rate Limiting and Timeouts

- Login Endpoint Rate Limiting: Implemented via database-persisted failed_attempts counter. Threshold: 5 failed attempts. Lockout duration: 15 minutes.
- No additional HTTP-level rate limiting is implemented in the current project scope. In production, Nginx limit_req is recommended.
- No explicit per-endpoint HTTP timeout policy is configured in the current development scope.

5. STATE, SESSION, AND CACHE IMPLEMENTATION

5.1 Session Data Structure

The Flask application uses server-side session management via Flask-Login. The session cookie stores only the user_id and is signed with the Flask SECRET_KEY. The following data is stored in the session context:

Session Key	Type	Purpose	Lifecycle
_user_id	Integer (str)	Flask-Login managed; stores authenticated user primary key	Created on login_user(); destroyed on logout_user()
_fresh	Boolean	Flask-Login flag indicating whether session was established by direct login	Set at login; checked by @fresh_login_required
_remember	String	Flask-Login flag for remember-me cookie behaviour	Set based on login form checkbox

No application-specific data (file data, encryption keys, passwords) is ever stored in the session.

5.2 Caching Strategy

The system does not implement application-level data caching. Each file upload invokes a fresh key derivation; each file download invokes a fresh decryption operation. This is the correct security behaviour: caching any intermediate cryptographic value would violate the zero-trust storage principle.

5.3 State Transition Diagrams

Authentication and Session State Machine:

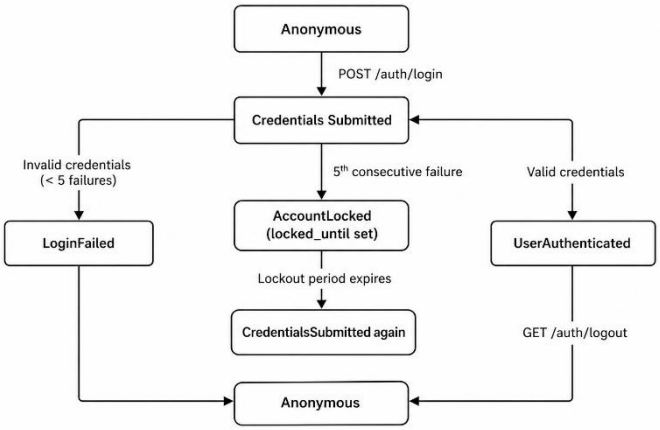


Figure 5.1 — Authentication and Session State Machine

6. SECURITY IMPLEMENTATION DETAILS

6.1 Authentication Tokens and Validation Logic

The system uses Flask-Login's session-cookie-based authentication. The session cookie is signed using HMAC-SHA1 with the application's SECRET_KEY. Flask validates this signature on every request before loading the user_id from the cookie.

Password validation rules enforced at registration:

- Minimum length: 8 characters
- The application does not impose a maximum length restriction (Bcrypt truncates at 72 bytes; this is documented in system notes for future enhancement).

6.2 Authorization and Role-Based Access Control (RBAC) Matrix

Feature / Route	User Role	Admin Role	Notes
Registration / Login	Yes	Yes	Public; no authentication required
Crypto Vault (own files)	Yes	Yes	User sees only their own FileRecord entries
File Upload (Encrypt)	Yes	Yes	Both roles can encrypt and store files
File Download (Decrypt)	Yes (own)	Yes (own)	Ownership verified against user_id on FileRecord
File Delete (Shred)	Yes (own)	Yes (own)	Ownership verified; cryptographic shredding applied
Admin Dashboard	No	Yes	Role check: current_user.role == 'admin'
System Audit Trail	No	Yes	Full AuditLog table accessible to Admin only
User Management Panel	No	Yes	Admin can view and delete user accounts
Admin User Deletion	No	Yes	Self-deletion is blocked at the route handler level

The screenshots below show the two role-based interfaces as implemented. Figure 6.1 shows the User dashboard (Crypto Vault), and Figure 6.2 shows the Admin dashboard (System Audit Trail).

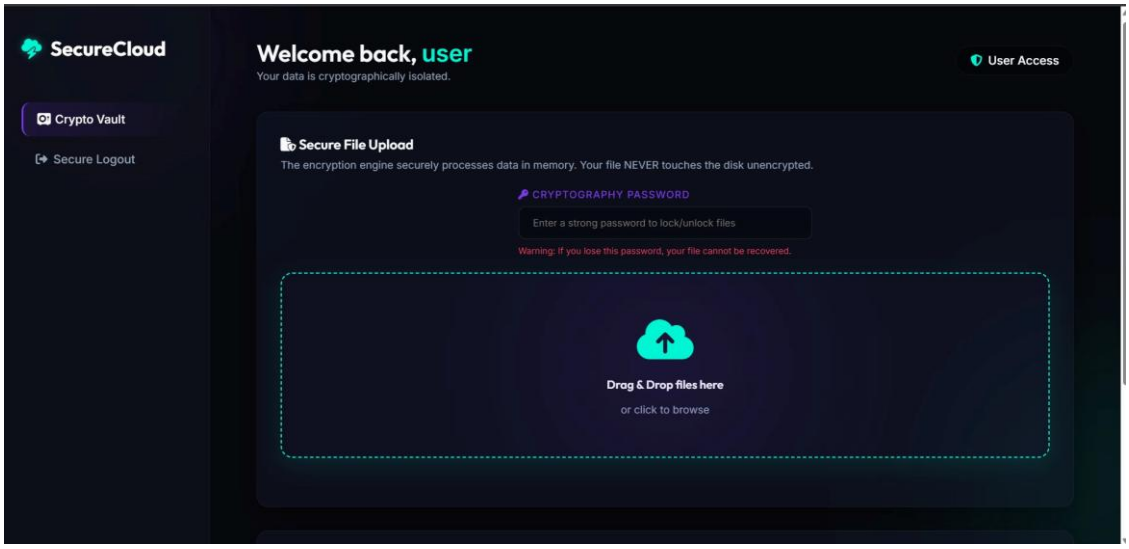


Figure 6.1 — User Dashboard: Crypto Vault with Role-Based Access (User Access)

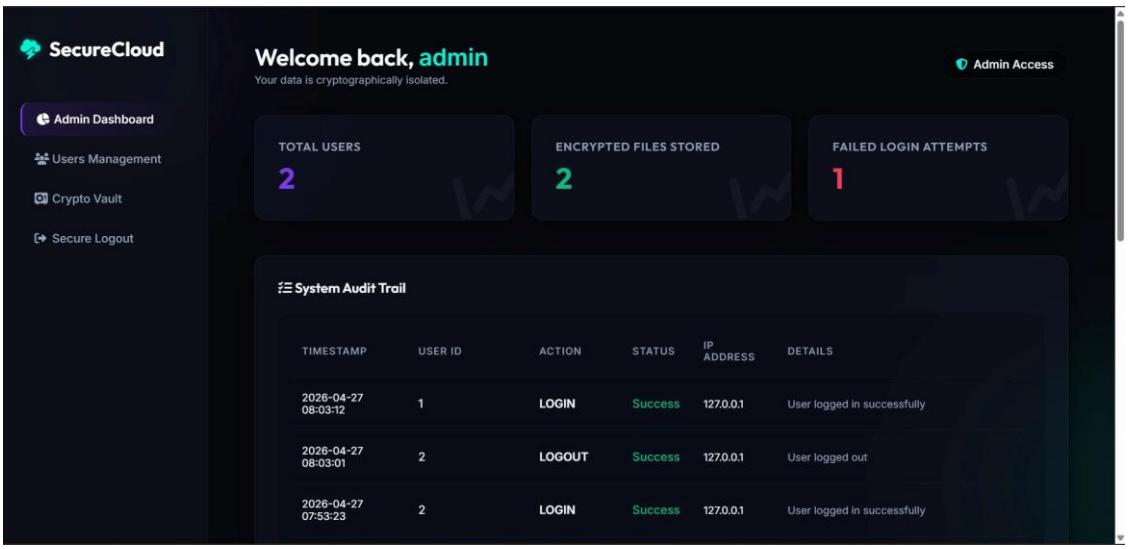


Figure 6.2 — Admin Dashboard: System Metrics and Audit Trail (Admin Access)

6.3 Data Encryption (At Rest and In Transit)

At Rest

- All stored files are encrypted with AES-256-GCM prior to any disk write.
- Encryption keys are derived per-file using PBKDF2HMAC-SHA256 with 600,000 iterations and a unique random 32-byte salt. Keys are never stored.
- Cryptographic metadata (salt, nonce) stored in the database is not sufficient to decrypt a file without the user's password.

In Transit

- In production: all traffic is served over HTTPS/TLS 1.3 via a reverse proxy with TLS termination.
- In the current development scope: HTTP is used on localhost. The encryption_password field in POST requests is in the request body (not query parameters) and is never logged.
- File downloads use send_file() from a BytesIO in-memory buffer, delivering decrypted content directly over the channel without disk intermediary.

6.4 Input Validation and Sanitization Rules

- username: Minimum 3 characters; maximum 80 characters; validated server-side. HTML special characters escaped by Jinja2 template rendering.
- password: Minimum 8 characters; validated server-side. Never echoed back in responses.
- encryption_password / decryption_password: No length restriction. Never logged or stored.
- file: Validated that request.files['file'] is present and non-empty. File size limit enforced by Flask's MAX_CONTENT_LENGTH (default: 16 MB).
- file_id URL parameter: Cast to integer using int(file_id) in route definition. Non-integer values return 404.

7. ERROR HANDLING, LOGGING, AND MONITORING

7.1 Standardized Error Message Formats

The system uses a consistent error response format for all AJAX endpoints:

```
{"status": "error", "message": "<human-readable description>", "code":
"<error_code>"}
```

Error Code	HTTP Status	Trigger Condition
AUTH_001	401	Request to protected route without active session
AUTH_002	403	Authenticated user accessing a route outside their role
FILE_001	400	Missing file or encryption password in upload request
FILE_002	400	Wrong decryption password or GCM authentication tag failure
FILE_003	404	FileRecord not found or user_id mismatch on access
FILE_004	500	I/O error during ciphertext read/write
DB_001	500	SQLAlchemy session commit failure
IPS_001	403	Login attempt on locked account

7.2 Application Logging Levels and Structure

The system implements two-tier logging:

Tier 1 — Security Audit Log (database): All security-relevant events are written to the audit_logs table. Fields: user_id, action (LOGIN/LOGOUT/UPLOAD/DOWNLOAD/DELETE), status (SUCCESS/FAILURE), ip_address, timestamp.

Tier 2 — Application Log (Python logging module):

- INFO: Normal operation events (server start, user registration, successful file operations)
- WARNING: IPS threshold approaching; unexpected but recoverable conditions
- ERROR: Exception conditions (I/O errors, database failures, decryption failures)
- DEBUG: Detailed trace information for development environments only; disabled in production

8. CONFIGURATION AND ENVIRONMENT SETUP

8.1 Environment Variables and Configuration Files

Variable	Purpose	Default / Behavior
FLASK_SECRET_KEY	Flask session signing key; must be cryptographically random	No default; application fails to start if absent in production mode
DATABASE_URL	SQLAlchemy database connection string	sqlite:///secure_retail.db (local file in project directory)
STORAGE_DIR	Absolute path to the encrypted file storage directory	Default: ./storage/ relative to application root
FLASK_ENV	Application environment mode	'development' or 'production'; controls debug output and error detail exposure
MAX_CONTENT_LENGTH	Maximum upload file size in bytes	Default: 16,777,216 (16 MB)
LOGIN_LOCKOUT_ATTEMPTS	Number of failed attempts before account lockout	Default: 5
LOGIN_LOCKOUT_MINUTES	Duration of account lockout in minutes	Default: 15

8.2 Feature Flags

Toggle	Environment Variable Value	Effect
Enable Debug Mode	FLASK_ENV=development	Enables Flask debug toolbar, verbose error pages, and auto-reloader
Disable Debug Mode	FLASK_ENV=production	Suppresses all internal error detail in HTTP responses; recommended for deployment
Bootstrap Admin Account	ADMIN_USERNAME and ADMIN_PASSWORD set	Creates admin user on first application startup if no admin exists in database
Increase Upload Limit	MAX_CONTENT_LENGTH=<bytes>	Raises the maximum permitted file upload size